

Practical 6

17ce049

Online Retails Purchase

Introduction:

Step 1. Import the necessary libraries

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# set the graphs to show in the jupyter notebook
%matplotlib inline

# set seaborn graphs to a better style
sns.set(style="ticks")
```

Step 2. Import the dataset from this [address](#)

(https://raw.githubusercontent.com/guipsamora/pandas_exercises/master/07_Visualization/Online_Retail/Online_Retail.csv)

Step 3. Assign it to a variable called online_rt

Note: if you receive a utf-8 decode error, set `encoding = 'latin1'` in `pd.read_csv()`.

In [2]:

```

path = 'https://raw.githubusercontent.com/guipsamora/pandas_exercises/master/07_Visualizati
online_rt = pd.read_csv(path, encoding = 'latin1')
online_rt.head()

```

Out[2]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	12/1/10 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	17850.0	United Kingdom

Step 4. Create a histogram with the 10 countries that have the most 'Quantity' ordered except UK

In [3]:

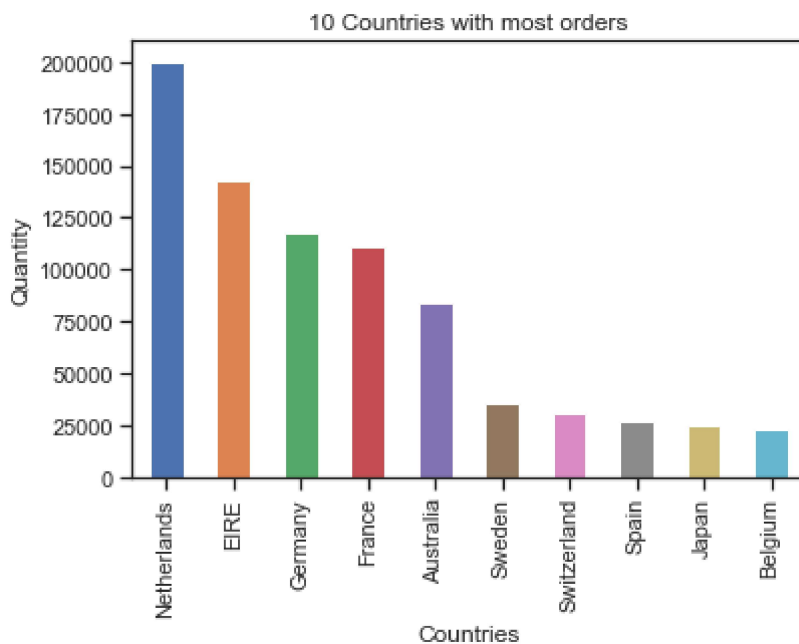
```
# group by the Country
countries = online_rt.groupby('Country').sum()

# sort the value and get the first 10 after UK
countries = countries.sort_values(by = 'Quantity',ascending = False)[1:11]

# create the plot
countries['Quantity'].plot(kind='bar')

# Set the title and labels
plt.xlabel('Countries')
plt.ylabel('Quantity')
plt.title('10 Countries with most orders')

# show the plot
plt.show()
```



Step 5. Exclude negative Quantity entries

In [4]:

```
online_rt = online_rt[online_rt.Quantity > 0]
online_rt.head()
```

Out[4]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	12/1/10 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	17850.0	United Kingdom

Step 6. Create a scatterplot with the Quantity per UnitPrice by CustomerID for the top 3 Countries (except UK)

In [5]:

```

# groupby CustomerID
customers = online_rt.groupby(['CustomerID', 'Country']).sum()

# there is an outlier with negative price
customers = customers[customers.UnitPrice > 0]

# get the value of the index and put in the column Country
customers['Country'] = customers.index.get_level_values(1)

# top three countries
top_countries = ['Netherlands', 'EIRE', 'Germany']

# filter the dataframe to just select ones in the top_countries
customers = customers[customers['Country'].isin(top_countries)]

#####
# Graph Section #
#####

# creates the FaceGrid
g = sns.FacetGrid(customers, col="Country")

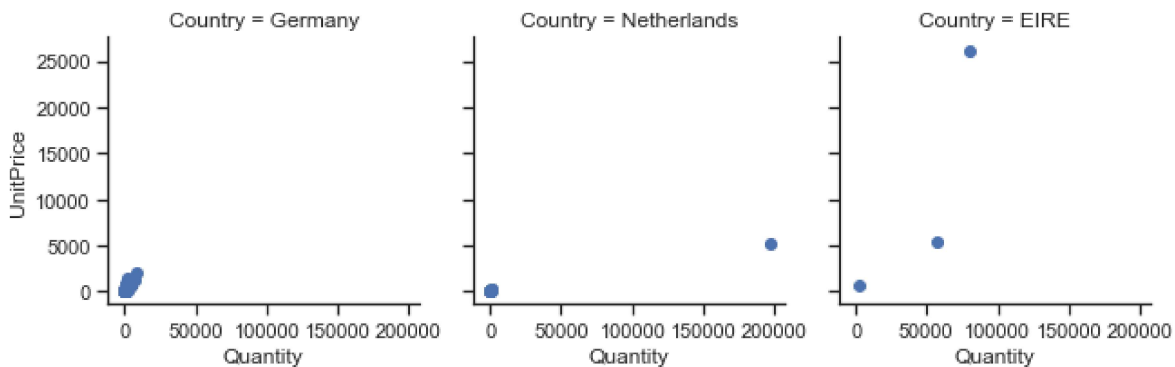
# map over a make a scatterplot
g.map(plt.scatter, "Quantity", "UnitPrice", alpha=1)

# adds Legend
g.add_legend()

```

Out[5]:

<seaborn.axisgrid.FacetGrid at 0x28c5e47e400>



Step 7. Investigate why the previous results look so uninformative.

This section might seem a bit tedious to go through. But I've thought of it as some kind of a simulation of problems one might encounter when dealing with data and other people. Besides there is a prize at the end (i.e. Section 8).

(But feel free to jump right ahead into Section 8 if you want; it doesn't require that you finish this section.)

Step 7.1 Look at the first line of code in Step 6. And try to figure out if it leads to any kind of problem.

Step 7.1.1 Display the first few rows of that DataFrame.

In [6]:

```
#This takes our initial dataframe groups it primarily by 'CustomerID' and secondarily by 'Country'
#It sums all the (non-indexical) columns that have numerical values under each group.
customers = online_rt.groupby(['CustomerID', 'Country']).sum().head()

#Here's what it looks like:
customers
```

Out[6]:

CustomerID	Country	Quantity	UnitPrice
12346.0	United Kingdom	74215	1.04
12347.0	Iceland	2458	481.21
12348.0	Finland	2341	178.71
12349.0	Italy	631	605.10
12350.0	Norway	197	65.30

Step 7.1.2 Think about what that piece of code does and display the dtype of UnitPrice

In [7]:

```
customers.UnitPrice.dtype
#So it's 'float64'
#But why did we sum 'UnitPrice', to begin with?
#If 'UnitPrice' wasn't something that we were interested in then it would be OK
#since we wouldn't care whether UnitPrice was being summed or not.
#But we want our graphs to reflect 'UnitPrice'!
#Note that summing up 'UnitPrice' can be highly misleading.
#It doesn't tell us much as to what the customer is doing.
#Suppose, a customer places one order of 1000 items that are worth $1 each.
#Another customer places a thousand orders of 1 item worth $1.
#There isn't much of a difference between what the former and the latter customers did.
#After all, they've spent the same amount of money.
#so we should be careful when we're summing columns. Sometimes we intend to sum just one column
#('Quantity' in this case) and another column like UnitPrice gets into the mix.
```

Out[7]:

```
dtype('float64')
```

Step 7.1.3 Pull data from online_rt for CustomerID s 12346.0 and 12347.0.

In [8]:

```
display(online_rt[online_rt.CustomerID == 12347.0].
        sort_values(by='UnitPrice', ascending = False).head())
display(online_rt[online_rt.CustomerID == 12346.0].
        sort_values(by='UnitPrice', ascending = False).head())
#The result is exactly what we'd suspected. Customer 12346.0 placed
#one giant order, whereas 12347.0 placed a lot of smaller orders.
#So we've identified one potential reason why our plots looked so weird at section 6.
#At this stage we need to go back to the initial problem we've specified at section 6.
#And make it more precise.
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Cou
428966	573511	22423	REGENCY CAKESTAND 3 TIER	6	10/31/11 12:25	12.75	12347.0	Ice
286637	562032	22423	REGENCY CAKESTAND 3 TIER	3	8/2/11 8:48	12.75	12347.0	Ice
72267	542237	22423	REGENCY CAKESTAND 3 TIER	3	1/26/11 14:30	12.75	12347.0	Ice
148300	549222	22423	REGENCY CAKESTAND 3 TIER	3	4/7/11 10:43	12.75	12347.0	Ice
428967	573511	23173	REGENCY TEAPOT ROSES	2	10/31/11 12:25	9.95	12347.0	Ice

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Count
61619	541431	23166	MEDIUM CERAMIC TOP STORAGE JAR	74215	1/18/11 10:01	1.04	12346.0	Unit Kingd

Step 7.2 Reinterpreting the initial problem.

To reiterate the question that we were dealing with:

"Create a scatterplot with the Quantity per UnitPrice by CustomerID for the top 3 Countries"

The question is open to a set of different interpretations. We need to disambiguate.

We could do a single plot by looking at all the data from the top 3 countries. Or we could do one plot per country. To keep things consistent with the rest of the exercise, let's stick to the latter option. So that's settled.

But "top 3 countries" with respect to what? Two answers suggest themselves: Total sales volume (i.e. total quantity sold) or total sales (i.e. revenue). This exercise goes for sales volume, so let's stick to that.

Step 7.2.1 Find out the top 3 countries in terms of sales volume.

In [9]:

```
sales_volume = online_rt.groupby('Country').Quantity.sum().sort_values(ascending=False)

top3 = sales_volume.index[1:4] #We are excluding UK
top3
```

Out[9]:

```
Index(['Netherlands', 'EIRE', 'Germany'], dtype='object', name='Country')
```

Step 7.2.2

Now that we have the top 3 countries, we can focus on the rest of the problem:

"Quantity per UnitPrice by CustomerID".

We need to unpack that.

"by CustomerID" part is easy. That means we're going to be plotting one dot per CustomerID's on our plot. In other words, we're going to be grouping by CustomerID.

"Quantity per UnitPrice" is trickier. Here's what we know:

*One axis will represent a Quantity assigned to a given customer. This is easy; we can just plot the total Quantity for each customer.

*The other axis will represent a UnitPrice assigned to a given customer. Remember a single customer can have any number of orders with different prices, so summing up prices isn't quite helpful. Besides it's not quite clear what we mean when we say "unit price per customer"; it sounds like price of the customer! A reasonable alternative is that we assign each customer the average amount each has paid per item. So let's settle that question in that manner.

Step 7.3 Modify, select and plot data

Step 7.3.1 Add a column to `online_rt` called `Revenue` calculate the revenue ($Quantity * UnitPrice$) from each sale.

We will use this later to figure out an average price per customer.

In [10]:

```
online_rt['Revenue'] = online_rt.Quantity * online_rt.UnitPrice
online_rt.head()
```

Out[10]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	12/1/10 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	17850.0	United Kingdom

Step 7.3.2 Group by CustomerID and Country and find out the average price (AvgPrice) each customer spends per unit.

In [11]:

```
grouped = online_rt[online_rt.Country.isin(top3)].groupby(['CustomerID', 'Country'])

plottable = grouped['Quantity', 'Revenue'].agg('sum')
plottable['AvgPrice'] = plottable.Revenue / plottable.Quantity

# get the value of the index and put in the column Country
plottable['Country'] = plottable.index.get_level_values(1)
plottable.head()
```

Out[11]:

		Quantity	Revenue	AvgPrice	Country
CustomerID	Country				
12426.0	Germany	258	582.73	2.258643	Germany
12427.0	Germany	533	825.80	1.549343	Germany
12468.0	Germany	366	729.54	1.993279	Germany
12471.0	Germany	8212	19824.05	2.414034	Germany
12472.0	Germany	4148	6572.11	1.584405	Germany

Step 7.3.3 Plot

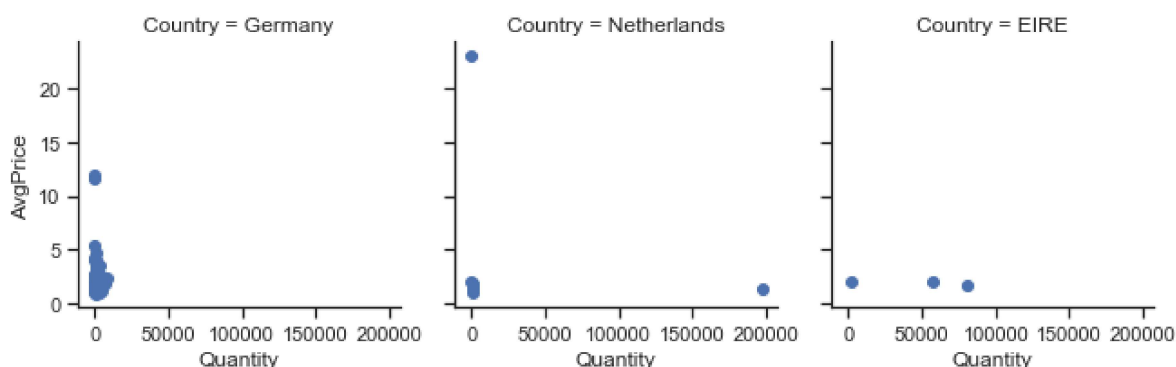
In [12]:

```
#####
# Graph Section v 2#
#####

# creates the FaceGrid
g = sns.FacetGrid(plottable, col="Country")

# map over a make a scatterplot
g.map(plt.scatter, "Quantity", "AvgPrice", alpha=1)

# adds Legend
g.add_legend();
```



Step 7.4 What to do now?

We aren't much better-off than what we started with. The data are still extremely scattered around and don't seem quite informative.

But we shouldn't despair! There are two things to realize: 1) The data seem to be skewed towards the axes (e.g. we don't have any values where Quantity = 50000 and AvgPrice = 5). So that might suggest a trend. 2) We have more data! We've only been looking at the data from 3 different countries and they are plotted on different graphs.

So: we should plot the data regardless of Country and hopefully see a less scattered graph.

Step 7.4.1 Plot the data for each CustomerID on a single graph

In [13]:

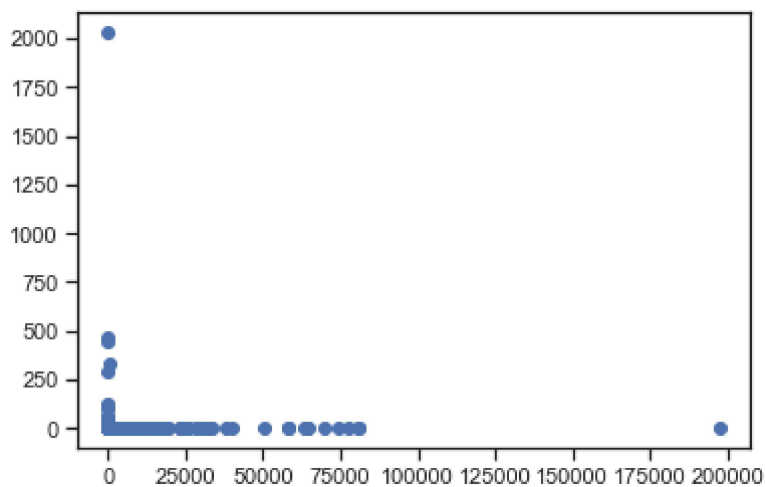
```
grouped = online_rt.groupby(['CustomerID'])
plottable = grouped['Quantity', 'Revenue'].agg('sum')
plottable['AvgPrice'] = plottable.Revenue / plottable.Quantity
```

```
# map over a make a scatterplot
plt.scatter(plottable.Quantity, plottable.AvgPrice)
plt.plot()
```

#Turns out the graph is still extremely skewed towards the axes like an exponential decay f

Out[13]:

[]



Step 7.4.2 Zoom in so we can see that curve more clearly

In [14]:

```
grouped = online_rt.groupby(['CustomerID', 'Country'])
plottable = grouped.agg({'Quantity': 'sum',
                        'Revenue': 'sum'})
plottable['AvgPrice'] = plottable.Revenue / plottable.Quantity

# map over a make a scatterplot
plt.scatter(plottable.Quantity, plottable.AvgPrice)

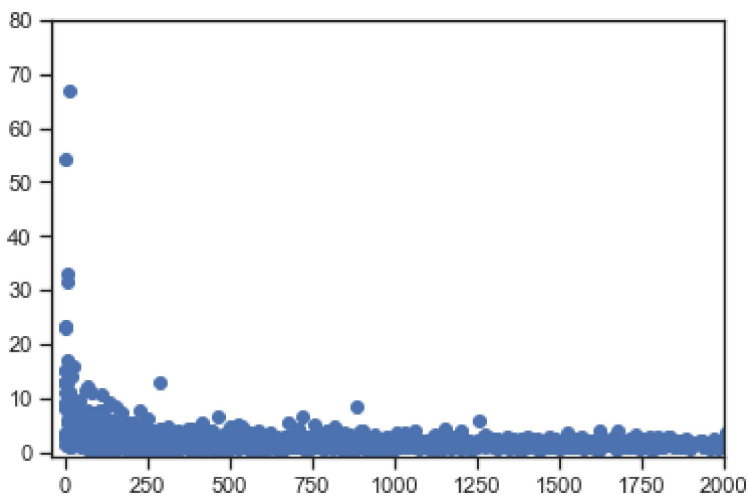
#Zooming in. (I'm starting the axes from a negative value so that
#the dots can be plotted in the graph completely.)
plt.xlim(-40, 2000)
plt.ylim(-1, 80)

plt.plot()

#And there is still that pattern, this time in close-up!
```

Out[14]:

[]



8. Plot a line chart showing revenue (y) per UnitPrice (x).

Did Step 7 give us any insights about the data? Sure! As average price increases, the quantity ordered decreases. But that's hardly surprising. It would be surprising if that wasn't the case!

Nevertheless the rate of drop in quantity is so drastic, it makes me wonder how our revenue changes with respect to item price. It would not be that surprising if it didn't change that much. But it would be interesting to know whether most of our revenue comes from expensive or inexpensive items, and how that relation looks like.

That is what we are going to do now.

8.1 Group UnitPrice by intervals of 1 for prices [0,50), and sum Quantity and Revenue .

In [15]:

```
#These are the values for the graph.
#They are used both in selecting data from
#the DataFrame and plotting the data so I've assigned
#them to variables to increase consistency and make things easier
#when playing with the variables.
price_start = 0
price_end = 50
price_interval = 1

#Creating the buckets to collect the data accordingly
buckets = np.arange(price_start,price_end,price_interval)

#Select the data and sum
revenue_per_price = online_rt.groupby(pd.cut(online_rt.UnitPrice, buckets)).Revenue.sum()
revenue_per_price.head()
```

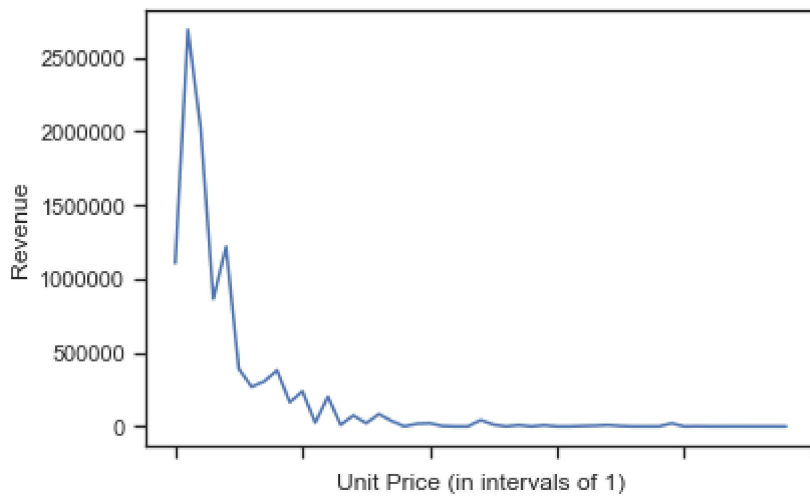
Out[15]:

```
UnitPrice
(0, 1]    1.107775e+06
(1, 2]    2.691765e+06
(2, 3]    2.024143e+06
(3, 4]    8.651018e+05
(4, 5]    1.219377e+06
Name: Revenue, dtype: float64
```

8.3 Plot.

In [16]:

```
revenue_per_price.plot()  
plt.xlabel('Unit Price (in intervals of '+str(price_interval)+'')  
plt.ylabel('Revenue')  
plt.show()
```



8.4 Make it look nicer.

x-axis needs values.

y-axis isn't that easy to read; show in terms of millions.

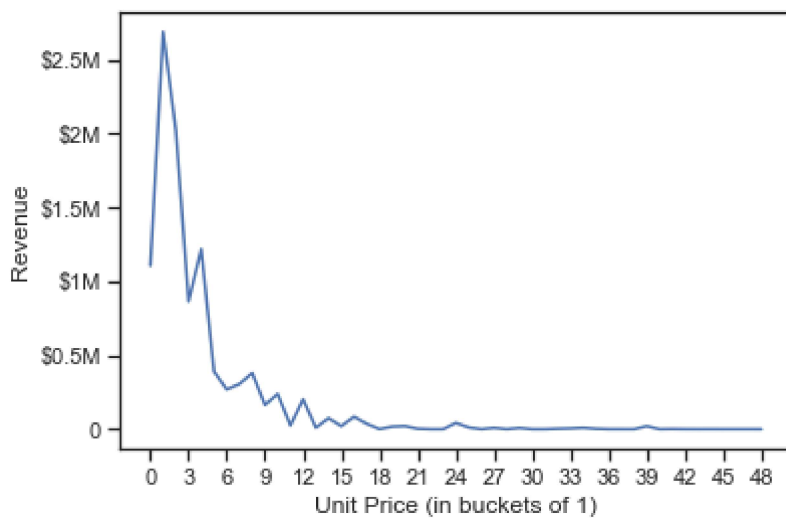
In [17]:

```
revenue_per_price.plot()

#Place Labels
plt.xlabel('Unit Price (in buckets of '+str(price_interval)+'')
plt.ylabel('Revenue')

#Even though the data is bucketed in intervals of 1,
#I'll plot ticks a little bit further apart from each other to avoid cluttering.
plt.xticks(np.arange(price_start,price_end,3),
           np.arange(price_start,price_end,3))
plt.yticks([0, 500000, 1000000, 1500000, 2000000, 2500000],
           ['0', '$0.5M', '$1M', '$1.5M', '$2M', '$2.5M'])
plt.show()

#Looks like a major chunk of our revenue comes from items worth $0-$3!
```



BONUS: Create your own question and answer it.

In []: